VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



Deep Learning and Its Applications (CO3133)

ASSIGNMENT 2

Five-Stage Sleep Classification from EEG

Final Report

Instructor: Dr. Le Thanh Sach

Class: CC01

Group: G-09

Students: Nguyen Hoang Quoc – 2353027
Nguyen Vu Long – 2352696
Le Phuoc Vu – 2353341

Ho Chi Minh City, 2026

Table of Contents

List of Figures **2**
List of Tables **3**
I Introduction **4**
1.1 Background and Motivation . . . 4
1.2 Problem Statement and Objectives . . . 4
1.3 Scope and Main Findings . . . 5
1.4 Dataset Rationale and Proposal-to-Final Scope . . . 5
II Dataset and Exploratory Analysis **5**
2.1 Source, Cohort and Annotation Mapping . . . 5
2.2 Inventory, Coverage and Integrity . . . 6
2.3 Stage Distribution and Imbalance . . . 7
2.4 Participant-Separated Splits . . . 7
2.5 Waveform Preprocessing . . . 9
III Methodology **9**
3.1 Shared Workflow . . . 9
3.2 Custom Convolutional Baseline . . . 11
3.3 Custom Temporal Transformer . . . 11
3.4 AttnSleep . . . 12
3.5 Pretrained CBraMod and Fine-Tuning . . . 13
3.6 Learning Objective . . . 14
IV Experimental Design **14**
4.1 Training Settings and Checkpoint Selection . . . 14
4.2 Evaluation Measures . . . 15
4.3 Three Distinct Result Summaries . . . 18
4.4 Comparison Hypotheses and Controlled Experiments . . . 18
V Experimental Results **20**
5.1 Five-Fold Development Performance . . . 20
5.2 Held-Out Participant Evaluation . . . 21
5.3 Learning Behaviour and Selected Epochs . . . 22
5.4 Controlled Ablation of Ensemble Aggregation . . . 23
5.5 Preparation-Recipe Sensitivity Results . . . 24
5.6 Model Size and Training Cost . . . 24
5.7 Inference Cost . . . 25
VI Analysis and Error Analysis **25**
6.1 Stage-Specific Strengths and Weaknesses . . . 25
6.2 Error Taxonomy . . . 26
6.3 Correct, Difficult-Correct and Failure Examples . . . 27
6.4 Interpretation of the Model Comparison . . . 27
VII Reproducibility and Limitations **29**
7.1 Attribution and Experiment Traceability . . . 29
7.2 Data and Experimental Limits . . . 29
VIII Conclusion **31**
AI Usage Disclosure **31**
A Participant Membership and Detailed Fold Results **32**
References **33**

List of Figures

1 Epoch counts per participant 6
2 Sleep-stage proportions by partition 7
3 Example EEG waveforms by stage 10
4 Training and evaluation workflow 10
5 Fold 1 learning curves 22
6 Held-out test confusion matrices 26
7 Prediction-linked waveform examples 28

List of Tables

1 Cohort inventory by partition 6
2 Sleep-stage counts by partition 7
3 Five-fold development splits 8
4 Main training configurations 14
5 Development performance across five folds 20
6 Validation precision and recall 20
7 Per-stage validation F1 20
8 Pooled validation performance 20
9 Per-stage pooled validation F1 21
10 Held-out test ensemble performance 21
11 Per-stage held-out test F1 21
12 Selected and completed training epochs 22
13 Transformer ensemble ablation 23
14 Preprocessing-recipe sensitivity 24
15 Model size and training cost 24
16 CPU inference cost 25
17 Largest Transformer error groups 26
18 Participant membership by partition 32
19 Detailed validation results by fold 32

I Introduction

1.1 Background and Motivation

Sleep staging assigns a physiological stage to each 30-second interval of a polysomnographic recording. Automating this process requires a model to distinguish local EEG patterns while handling variation between people and unequal stage frequencies. This study compares a simple convolutional baseline, a compact temporal Transformer, a task-specific attention model, and a pretrained EEG foundation model. The aim is to identify a useful balance between classification quality, model size and training cost under a participant-separated evaluation protocol. The task is retrospective sleep-stage classification, not diagnosis or a clinically validated scoring service.

1.1.1 EEG and the Five-Stage Task

Electroencephalography (EEG) records voltage differences over time. Here, Fpz-Cz and Pz-Oz are two bipolar derivations: each measures a difference between a pair of scalp electrode positions. They are two signal channels, not two independent subjects or a dense spatial map of the brain. A sampled waveform preserves amplitude and temporal order; a spectrogram instead represents energy over time and frequency. This study learns from the waveform, although a model may extract frequency-related features internally.

Sleep alternates between non-REM and REM phases. N1 describes the transition into sleep, N2 is a further non-REM stage, and N3 is slow-wave sleep. REM brain activity can resemble wakefulness, while eye movement and muscle signals also inform physiological staging [1]. This motivates learning both short events and longer within-epoch relationships. It also explains why a two-channel EEG-only task may remain ambiguous: the classifier does not receive every modality available in the original polysomnogram. These physiological distinctions motivate the experiment; they are not evidence that a particular learned filter detects a clinically identified event.

1.2 Problem Statement and Objectives

For each labelled epoch, the common preprocessed input and target are

$$X_i \in \mathbb{R}^{2\times 30\times 200}, \qquad y_i \in \{0,1,2,3,4\} = \{\mathrm{W}, \mathrm{N1}, \mathrm{N2}, \mathrm{N3}, \mathrm{REM}\}. \tag{1}$$

The axes represent EEG derivation, second, and sample within that second. A model produces five logits and selects the stage with the largest value. The research question is: *How do compact waveform models and a fine-tuned EEG foundation model compare on unseen participants, and what accuracy–cost trade-offs do they offer?*

The objectives are to establish a reproducible full-cohort data representation, compare four modelling approaches on shared participant partitions, report both overall and stage-specific performance, and investigate whether a common preprocessing recipe is suitable for the task-specific attention model. Accuracy and macro-F1 are the primary reporting measures required for classification; macro-F1 also controls checkpoint selection [2].

1.3 Scope and Main Findings

The completed AttnSleep rerun supplies full checkpoint and probability evidence and replaces its earlier aggregate-only main result. The final comparison uses all 195,469 eligible epochs from 78 Sleep Cassette participants. It does not use the earlier 6,474-epoch pilot subset. A custom CNN and temporal Transformer are trained from scratch, AttnSleep is reproduced with an input adapter, and CBraMod is fine-tuned from released pretrained weights. All four complete five development folds and a separate five-model soft-voting evaluation on 12 held-out participants.

The temporal Transformer achieves the highest observed test macro-F1, 0.7564, followed by AttnSleep at 0.7506, CBraMod at 0.7422, and the CNN at 0.7063. AttnSleep has the lowest logged training cost. These are within-project comparisons of complete model configurations; they neither isolate pretraining as a causal factor nor establish superiority over published benchmarks using different protocols.

1.4 Dataset Rationale and Proposal-to-Final Scope

Sleep Cassette provides expert annotations, five clinically meaningful target categories, repeated overnight recordings and enough epochs to evaluate deep models beyond a small image benchmark. Its difficulty is not defined by sample count alone: consecutive epochs are correlated, participants contribute unequal amounts of data, and uncommon or transitional stages can be hard to distinguish. The participant is therefore the unit of separation and uncertainty analysis. The public source, version, license, label conversion and cohort inventory are documented in Section II.

The approved dataset proposal began with EEG-derived representations and included a smaller pilot to control cost. The final comparison returns to the full eligible cohort, keeps the reserved participant membership, and uses a common waveform representation to accommodate both compact and pretrained EEG models. No participant or epoch is retained because a model predicts it correctly. The earlier 83-epoch-per-person quota is disabled. Four configurations require twenty development-fold fits; measured runtime and an inference-only ensemble ablation provide the final compute evidence. The scope change is in representation, model selection and evaluation procedure, not in the five-stage target or approved Sleep Cassette cohort.

II Dataset and Exploratory Analysis

2.1 Source, Cohort and Annotation Mapping

The data are Sleep-EDF Database Expanded, PhysioNet version 1.0.0 (DOI: 10.13026/C2X676). The archive is distributed under the Open Data Commons Attribution License v1.0 [3, 4, 5]. The approved cohort is Sleep Cassette: 78 participants and 153 nights, excluding Sleep Telemetry. The dataset proposal received approval without conditions. PSG waveforms are supplied in EDF and expert annotations in EDF+. The two EEG derivations used are Fpz-Cz and Pz-Oz, originally sampled at 100 Hz. EOG and EMG

are not model inputs.

Expert labels follow the legacy Rechtschaffen–Kales convention. Stages 3 and 4 are merged into N3; W, stage 1, stage 2 and R map to W, N1, N2 and REM. Movement and unscored intervals are discarded. This conversion provides five classes without claiming a new expert rescoring. Each retained example is a complete, non-overlapping 30-second interval. Long wake periods are limited to 30 minutes before the first scored sleep interval and 30 minutes after the last, within the recording boundaries. Because this cropping uses reference annotations, the experiment is a retrospective benchmark.

2.2 Inventory, Coverage and Integrity

The full waveform collection contains 195,469 epochs across all 153 nights. Participant contributions range from 824 to 4,985 epochs, with median 2,305.5; therefore epoch-level scores give greater weight to people with more retained data. The tensor and metadata occupy approximately 9.39 GB in decimal units. Recordings are paired with their hypnograms, with checks for channel identities, sampling frequency, timing consistency, complete epoch boundaries and archive checksums. Finite-signal checks are not equivalent to comprehensive physiological artefact removal.

Partition	Participants	Nights	Epochs
Development	66	130	170,537
Held-out test	12	23	24,932
Total	78	153	195,469

Table 1: *Participant, recording and epoch counts by partition.*

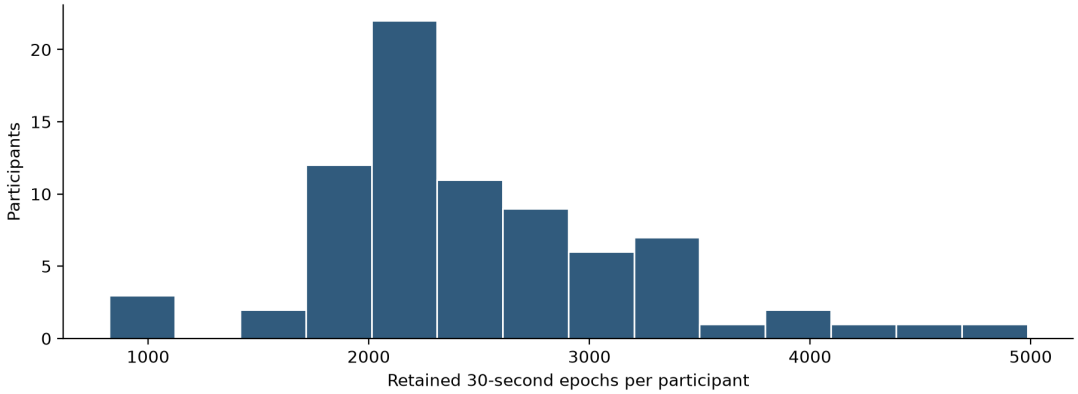


Figure 1: *Retained epochs per participant in the full cohort.*

2.3 Stage Distribution and Imbalance

Stage	Full cohort	Full (%)	Development	Test
W	65,941	33.73	59,966	5,975
N1	21,522	11.01	19,441	2,081
N2	69,132	35.37	58,754	10,378
N3	13,039	6.67	10,529	2,510
REM	25,835	13.22	21,847	3,988
Total	195,469	100.00	170,537	24,932

Table 2: *Class counts in the full cohort, development and test sets.*

N2 is the largest class in the full cohort (35.37%), and N3 is the smallest (6.67%), a count ratio of approximately 5.30:1. Development has more W than N2, whereas test has 41.62% N2 and 23.96% W. Thus the observed distribution differs across participant groups. Class weighting is estimated from each training fold only; validation and test frequencies are preserved. Macro-F1 and per-stage scores reveal failures that accuracy alone can conceal.

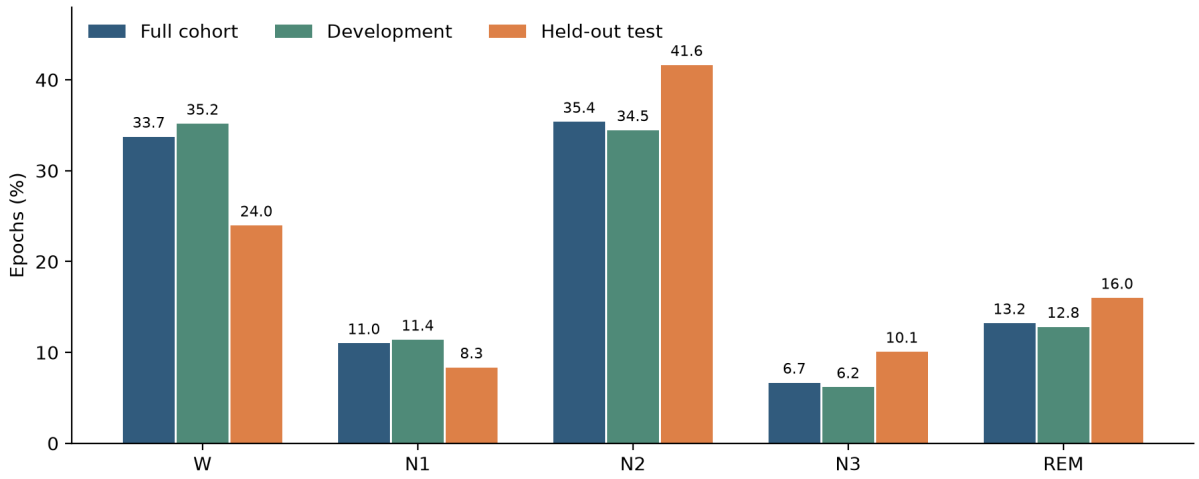


Figure 2: *Stage proportions after annotation-based wake trimming, by partition.*

2.4 Participant-Separated Splits

Twelve participants are held out for testing. The remaining 66 are divided into five stratified group folds, with shuffling and random state 42. A group is a participant, not an epoch or a night, so two nights from one person cannot appear on both sides of a fold. Each development epoch appears in validation exactly once and in training four times. The same reconstructed participant memberships and validation class supports are shared by the four main experiments.

2.4.1 Why Stratification Must Respect Groups

An epoch-wise random split can place highly related intervals from the same person in training and validation. A model may then benefit from participant-specific signal characteristics rather than demonstrate transfer to a new person. Ordinary stratification addresses class proportions but does not prevent this overlap. Grouped stratification combines a hard participant-separation constraint with an attempt to preserve stage frequencies as closely as the indivisible groups allow [6].

Let g_i be the participant of epoch i, $\mathcal{D}_{\text{dev}}$ the development set, and $\mathcal{G}_1, \ldots, \mathcal{G}_5$ disjoint sets of development participants. For fold k,

$$\mathcal{V}_k = \{(X_i, y_i) \in \mathcal{D}_{\text{dev}} : g_i \in \mathcal{G}_k\}, \tag{2}$$

$$\mathcal{T}_k = \mathcal{D}_{\text{dev}} \setminus \mathcal{V}_k, \qquad \mathcal{G}_k \cap \mathcal{G}_j = \varnothing \quad (k \neq j). \tag{3}$$

The validation sets partition development: $\bigcup_k \mathcal{V}_k = \mathcal{D}_{\text{dev}}$. If n_{kc} denotes validation examples of stage c in fold k, stratification seeks comparable ratios $n_{kc}/|\mathcal{V}_k|$ across folds. Exact ratios are not guaranteed because all epochs and nights of one participant must move together. Group isolation takes priority over equal fold sizes.

Fold	Train people	Val. people	Train epochs	Val. epochs
1	53	13	137,438	33,099
2	53	13	136,763	33,774
3	53	13	136,914	33,623
4	52	14	136,026	34,511
5	53	13	135,007	35,530

Table 3: *Participant-separated five-fold development splits.*

The original test membership is fixed before cross-validation. Stratification approximates stage balance subject to participant grouping; it does not make the folds equal in size. The validation data select both the epoch and stopping time, so pooled validation is a development estimate with selection optimism. It is not a set of untouched outer test folds. The held-out participant set provides a separate generalization assessment, although it has been inspected during project development and should not be described as a pristine, once-only final test.

2.4.2 Leakage Controls and Remaining Dependence

The held-out group is constructed by sorting participant identifiers, shuffling with seed 42, and reserving twelve people. Saved participant assignments, rather than repeated model-specific splitting, define the final protocol. Checks verify disjoint train/validation/test participant sets, unique recording–onset identities, and one validation assignment for each development epoch. Epoch identity checks do not constitute an exhaustive search for identical signal content across the source archive.

Class weights are computed from $\mathcal{T}_k$ only. Batch-normalization running statistics are learned during training and held fixed during validation and test. The common physical-unit conversion and fixed filter do not estimate population statistics from evaluation data;

there is no fitted whole-dataset z-score. Continuous filtering can use neighbouring times within a recording, but complete participant separation prevents a filtered recording from crossing a train/evaluation boundary. Reference-based wake cropping remains an explicit retrospective assumption.

Five folds offer a feasible estimate across several unseen-participant groups while retaining 52–53 people for each fit. This is neither leave-one-subject-out evaluation nor a reproduction of an author's 20-fold protocol. Fold results are correlated because training sets overlap, and validation labels influence checkpoint selection. Grouping addresses participant overlap; it does not make hyperparameter selection unbiased. A new untouched cohort or nested group evaluation would be needed for stronger confirmatory model-selection claims.

2.5 Waveform Preprocessing

The common signal preparation is:

1. Select eligible 30-second epochs using annotation mapping and wake margins, retaining recording identifiers and onsets. All eligible epochs are used; no participant subsampling is applied.
2. Apply a continuous, zero-phase 0.3–35 Hz FIR bandpass with a Hamming window before extracting epochs. Boundary padding is applied at recording edges.
3. Resample from 100 to 200 Hz using polyphase resampling with a Kaiser window. This matches the pretrained interface; it does not add measured high-frequency information.
4. Express amplitudes as microvolts divided by 100. A signal in volts is multiplied by 10^4. The scale is applied once, with no additional population z-scoring.
5. Reshape each two-channel, 6,000-sample epoch into $(2, 30, 200)$ while retaining chronological order. Store float32 waveforms, integer targets, participant membership and epoch metadata.

There is no augmentation, notch filter, stage oversampling or explicit amplitude-threshold artefact rejection in the reported main runs. The zero-phase filter uses surrounding signal within a recording, so these experiments do not establish causal online operation. A typical batch is $(64, 2, 30, 200)$ with 64 integer labels and output $(64, 5)$. Neither waveform axis is a frequency axis; the models are not trained on spectrogram images.

III Methodology

3.1 Shared Workflow

Waveforms remain memory-mapped on CPU and are transferred to the accelerator in batches. Development and test subsets use the stored participant partition. Classifiers return unnormalized logits; softmax is applied for probability averaging at test time. No sequence smoothing or neighbouring-epoch post-processing is used.

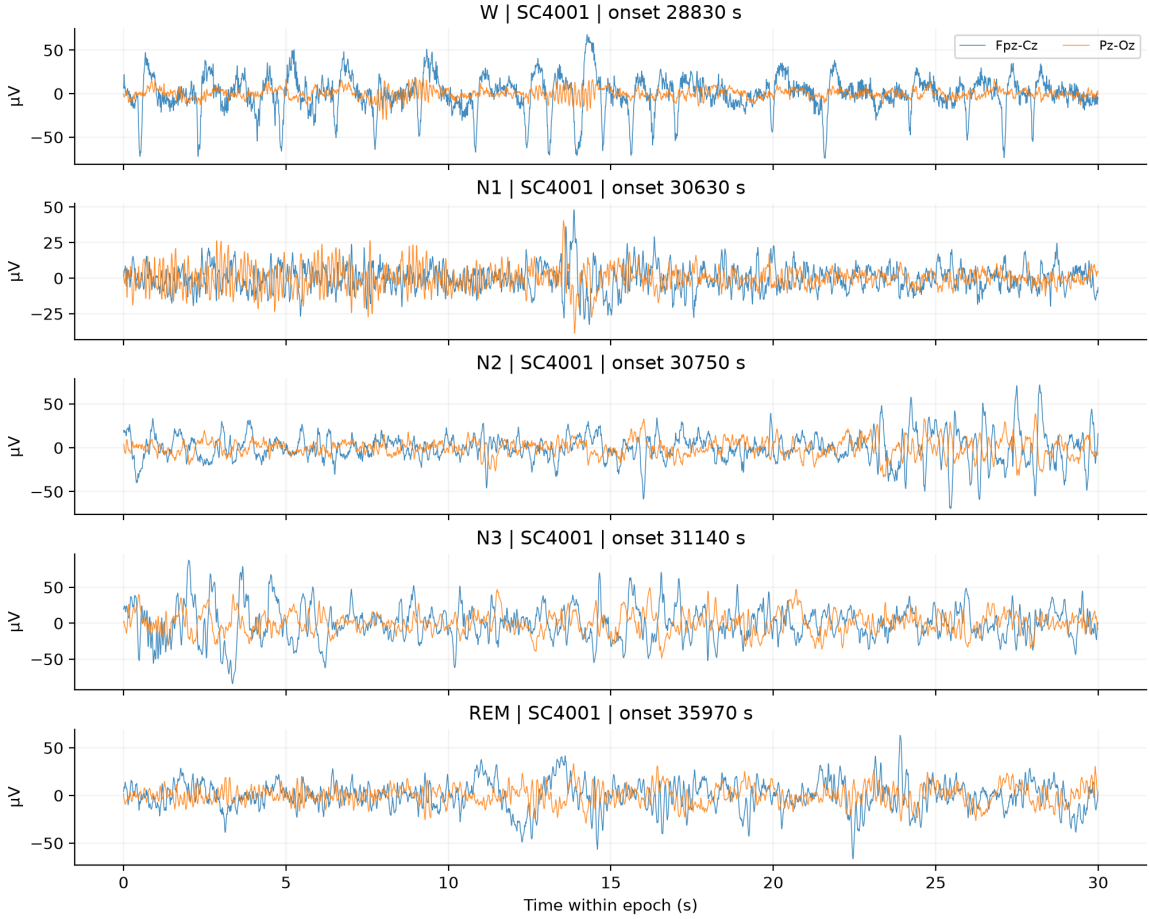


Figure 3: *First stored development epoch per stage, shown in microvolts.*

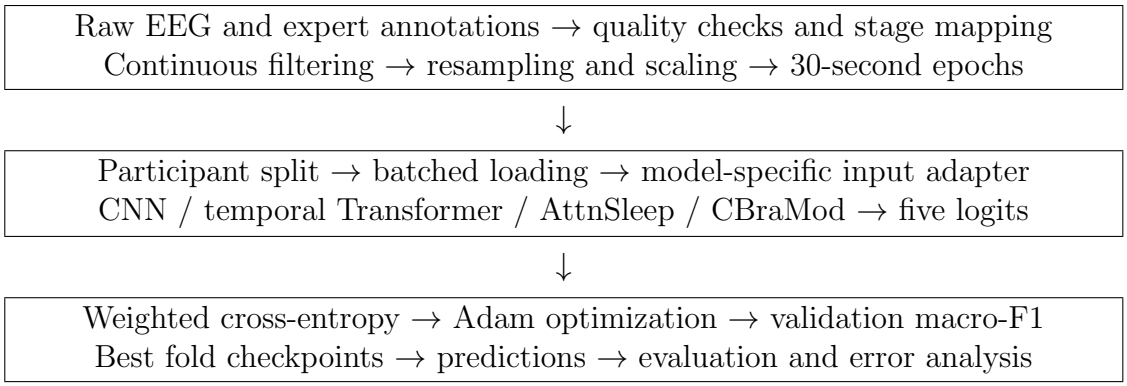


Figure 4: *Shared training workflow and five-model test soft voting.*

3.2 Custom Convolutional Baseline

3.2.1 Convolution, Pooling and Inductive Bias

A convolutional neural network applies the same learned local filters at different input positions. Local connectivity and shared weights allow recurring waveform patterns to be represented with fewer parameters than a fully connected mapping over every input value [7]. A schematic one-dimensional feature map is

$$h_j(t) = \phi\left(b_j + \sum_{c=1}^{C}\sum_{u=0}^{L-1} W_{jcu}x_c(t+u)\right), \tag{4}$$

where L is kernel length, c indexes input channels, j indexes output filters, and ϕ is an activation. Stride changes the spacing between evaluated positions; stacking layers expands the receptive field. Pooling reduces resolution, trading some timing detail for a smaller representation. The two-dimensional baseline applies the analogous operation over both grid axes.

Batch normalization learns an affine rescaling after normalizing training activations, while ReLU introduces nonlinearity. Dropout regularizes the classifier by randomly suppressing activations during training and is disabled for evaluation. These components support optimization and regularization; none guarantees generalization to a new participant.

3.2.2 Instantiated Baseline and Its Limitations

The CNN contains four 3×3 convolution blocks with widths 32, 64, 128 and 256. Each uses batch normalization, ReLU and 2×2 max pooling. For a $(2, 30, 200)$ input, the feature grids become $(32, 15, 100)$, $(64, 7, 50)$, $(128, 3, 25)$ and $(256, 1, 12)$. Global average pooling feeds a $256 \to 512 \to 5$ head with ReLU and dropout 0.5. All 523,237 parameters are trained from scratch.

This is a deliberately simple reference. Its two-dimensional neighbourhood spans within-second positions and corresponding positions in adjacent seconds, rather than time and frequency. Repeated pooling removes fine temporal detail. The baseline tests the usefulness of local waveform-grid patterns; it is not an optimized state-of-the-art sleep-staging architecture.

3.3 Custom Temporal Transformer

3.3.1 Waveform Tokenization

Each second is jointly encoded from the two channels by two one-dimensional convolution branches. Initial kernels of 25 and 101 samples represent short and longer temporal contexts, with strides 2 and 4. Both branches then use a 5-sample convolution with stride 2, batch normalization and GELU, followed by four ordered pooling bins. Their concatenated 512 features are projected to a 128-dimensional token. This multi-resolution idea is inspired by AttnSleep, but the tokenization and Transformer are custom components [8].

The resulting 30 chronological tokens receive learned temporal positional embeddings and a learned classification token:

$$(B, 2, 30, 200) \to (B, 30, 128) \to (B, 31, 128) \to (B, 5). \tag{5}$$

Three encoder layers use four attention heads and feed-forward width 384. Residual connections, layer normalization and dropout 0.1 are used. The normalized classification token feeds a five-logit head. There are 595,973 trainable parameters. Attention operates across the 30 seconds within an epoch; it does not use preceding or following sleep epochs. The one-second convolution boundary can limit features spanning adjacent patches, although attention subsequently connects their representations.

3.3.2 Self-Attention and Temporal Position

For token matrix $H \in \mathbb{R}^{31\times128}$, each attention head forms learned queries, keys and values. The standard scaled dot-product operation is [9]

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \qquad \text{Attention}(H) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V, \tag{6}$$

with $d_h = 32$ for four heads. Row-wise softmax gives normalized weights over source tokens; concatenated head outputs are projected back to width 128. Unlike a fixed convolutional neighbourhood, these weights depend on the current input and can connect distant seconds within the epoch.

In the implemented pre-normalized encoder, one layer computes

$$U = H + \text{MHA}(\text{LN}(H)), \tag{7}$$

$$H' = U + \text{MLP}(\text{LN}(U)). \tag{8}$$

The MLP uses two linear maps with GELU and dropout. Residual additions preserve a path for information and gradients. Learned positional vectors distinguish the ordered seconds; attention alone does not encode their chronological positions. A learned classification token aggregates evidence and its final representation supplies the classifier.

Using 31 tokens, including the classification token, keeps the attention matrix small. The attention interaction term grows quadratically with token count, so directly attending to all 6,000 time samples would be substantially more expensive. This is an architectural motivation, not a measured speedup against an unimplemented sample-level model. Attention weights also should not be interpreted as validated physiological explanations without additional analysis.

3.4 AttnSleep

AttnSleep combines a multi-resolution CNN, adaptive feature recalibration and a temporal context encoder [10, 8]. The adopted Sleep-EDF architecture retains the author's two temporal convolution branches, with initial kernels 50 and 400 at 100 Hz, a squeeze-and-excitation recalibration block, and two context-encoder layers. Its context module uses causal convolutions and five-head attention; a flattened representation feeds five output logits. The model has 522,805 trainable parameters and is trained from scratch.

The input adapter selects Fpz-Cz and converts the common 200 Hz representation back to 100 Hz, giving $(B, 1, 3000)$. Selecting every second sample is appropriate here because the common representation was filtered below 35 Hz before upsampling. This does not constitute a general resampling rule for arbitrary 200 Hz recordings. The stored amplitude scale is preserved. Thus all models share epoch identities and labels, but AttnSleep uses one channel at an effective 100 Hz whereas the other three use both channels at 200 Hz.

3.5 Pretrained CBraMod and Fine-Tuning

CBraMod is an EEG foundation model pretrained by masked patch reconstruction on a large EEG corpus [11, 12]. The released pretrained feature extractor is reused, while the reconstruction projection is replaced with a task-specific classifier. Each input has two channels and 30 one-second segments, giving 60 patches of 200 values.

A temporal convolution branch and a projected magnitude spectrum from a 200-point real FFT jointly embed each patch into 200 features. A depthwise convolution provides asymmetric conditional positional encoding. Twelve criss-cross layers model spatial and temporal dependencies separately: attention across channels at a fixed time and attention across time within a channel. The feed-forward width is 800. The external representation remains a waveform, although spectral features are derived internally.

The task head flattens the patch representation into 12,000 values and applies $12,000 \to 800 \to 200 \to 5$ linear layers, ELU and dropout 0.5. The backbone and new head are fine-tuned together; the encoder is not frozen. The model has 14,645,605 trainable parameters plus 200 fixed mask parameters. The head alone has 9,762,005 parameters, about two-thirds of the trainable model, which is relevant to both storage and regularization. FFT patch embedding stays in float32 under mixed precision. This is a downstream adaptation of the authors' backbone, not new foundation-model pretraining.

3.5.1 Transfer Learning and Criss-Cross Dependencies

Pretraining learns a representation before the supervised sleep task; fine-tuning adapts that representation and the newly initialized task head using labelled training participants. With full fine-tuning, gradients update both parts. A frozen-encoder alternative would update only the head, but it was not run as a controlled comparison here. Consequently, the present results do not measure the isolated benefit of unfreezing or of pretraining.

Conceptually, patch features form a channel-by-time grid $H \in \mathbb{R}^{C \times P \times d}$. Spatial attention relates $H_{:,p,:}$ at a fixed second, while temporal attention relates $H_{c,:,:}$ within a channel. Criss-cross modelling preserves these distinct relationships rather than treating all channel–time patches as an undifferentiated sequence [11]. In this experiment $C = 2$ and $P = 30$, so spatial context is limited to two bipolar derivations. Transfer across montage, signal statistics and task remains empirical: a larger pretrained model is not guaranteed to outperform a smaller specialized classifier.

3.6 Learning Objective

All four models minimize weighted multiclass cross-entropy. For a batch $\mathcal{B}$ with logits z_i, the optimized mean is

$$\mathcal{L}_{\mathcal{B}} = \frac{\sum_{i\in\mathcal{B}} w_{y_i}\,[-\log \operatorname{softmax}(z_i)_{y_i}]}{\sum_{i\in\mathcal{B}} w_{y_i}}. \tag{9}$$

Weights follow the adopted AttnSleep rule, using only counts n_c from the training portion of the current fold:

$$w_c = \operatorname{round}_2\left[\mu_c \max\left(\log \frac{\mu_c N_{\text{train}}}{n_c}, 1\right)\right], \qquad \boldsymbol{\mu} = (1.5, 2, 1.5, 1, 1.5)/5. \tag{10}$$

The same rule is used for all models. It was not retuned independently for each architecture. Targets are mutually exclusive integer classes; binary cross-entropy is not used.

The historical CNN, Transformer and CBraMod loss logs aggregate batch-weighted means by batch size. With unequal class weights, that historical aggregation is not exactly the globally weight-normalized epoch loss. The completed AttnSleep rerun uses the corrected sum-of-target-weights normalization. Accordingly, model selection and the reported comparisons rely on prediction-derived macro-F1, not small loss differences. Exact global weighted-loss reporting requires accumulating the weighted loss numerator and the sum of target weights over the full epoch. This logging limitation does not change the optimized batch objective or the reported confusion-matrix metrics.

IV Experimental Design

4.1 Training Settings and Checkpoint Selection

Model	Initial LR	Batch	Weight decay	Initialization
Custom CNN	10^{-5}	64	10^{-4}	Random
Transformer	3×10^{-5}	64	10^{-4}	Random
AttnSleep	10^{-3}	128	10^{-3}	Random
CBraMod	3×10^{-5}	128	10^{-4}	Pretrained backbone

Table 4: *Main training configurations.*

Every fold initializes a new model and optimizer. Adam uses AMSGrad; the maximum duration is 80 epochs. CNN, Transformer and CBraMod use cosine decay over 80 epochs to 10^{-6}. AttnSleep reduces its initial learning rate by a factor of ten after epoch 10. The checkpoint is the epoch with the highest validation macro-F1. Early stopping uses patience 10 and minimum improvement 0.001; checkpoint selection still accepts any strict macro-F1 improvement. All metrics reported for a fold refer to the same restored checkpoint, rather than separate best values for each metric.

Hyperparameters were chosen manually during project development, with an author-informed schedule for AttnSleep and lower learning rates for the other configurations.

There was no exhaustive, equal-budget hyperparameter search and no complete search ledger. The final configurations are therefore compared as trained systems, rather than estimates of each architecture's best achievable performance.

The main runs use an NVIDIA RTX 3090, as confirmed by the experiment owner. CUDA mixed precision, four data-loading workers, pinned memory and two-batch prefetching are enabled. PyTorch and NumPy seeds are 42, as is grouped splitting. Seeds are initialized before cross-validation, rather than independently reset per fold. The runs therefore share a reproducible starting convention but do not establish bitwise reproducibility or independent repeated-seed uncertainty.

4.2 Evaluation Measures

All measures use the fixed stage order W, N1, N2, N3, REM. Let $M \in \mathbb{N}^{5\times5}$ be the confusion matrix, with true stages in rows and predictions in columns. With $K = 5$ classes and $N = \sum_{a,b} M_{ab}$ epochs, define true support $n_c = \sum_b M_{cb}$ and predicted support $\hat{n}_c = \sum_a M_{ac}$. The one-versus-rest counts for stage c are

$$TP_c = M_{cc}, \quad FN_c = n_c - TP_c, \quad FP_c = \hat{n}_c - TP_c, \quad TN_c = N - TP_c - FN_c - FP_c. \tag{11}$$

For example, an N2 epoch predicted as N1 is a false negative for N2 and a false positive for N1. It is counted once as a multiclass error. Metrics are computed from all predictions in the evaluated partition, not averaged across minibatches. Training loss weights are not applied to evaluation counts.

4.2.1 Accuracy and Balanced Accuracy

Definition. Accuracy measures the fraction of all epochs assigned the correct stage. Balanced accuracy (BAcc) instead gives equal importance to the recall of each stage [13]:

$$\mathrm{Acc} = \frac{\sum_{c=1}^{K} TP_c}{N}, \qquad \mathrm{BAcc} = \frac{1}{K}\sum_{c=1}^{K} \frac{TP_c}{TP_c + FN_c}. \tag{12}$$

Both lie between zero and one, with larger values preferred. The unadjusted BAcc used here is 0.20 for a constant-stage predictor when all five stages occur. Accuracy weights stages by their actual support; BAcc gives each stage one-fifth of the total weight.

Why both are reported. Accuracy is an intuitive measure of total correct decisions and is required alongside macro-F1 by the classification rubric [2]. However, the largest class has about 5.30 times as many examples as the smallest. BAcc exposes low recall that accuracy can hide in a common-stage-dominated dataset. It does not directly penalize low precision for a specific stage, so it must be read with F1 and precision.

The report does not average one-versus-rest accuracies $(TP_c + TN_c)/N$. Their true-negative counts repeatedly include the same epochs and can produce deceptively large values. For single-label classification, their mean is $1 - 2(1 - \mathrm{Acc})/K$; at $K = 5$, the Transformer's actual test accuracy 0.7989 would become approximately 0.9195 under that different definition. The latter is not the fraction of correctly staged epochs.

4.2.2 Macro-Averaged Precision

Definition. Precision measures how often a predicted stage is correct:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Macro-Precision} = \frac{1}{K}\sum_{c=1}^{K} P_c. \tag{13}$$

The macro average treats each stage equally rather than weighting it by frequency [14]. Precision is included to reveal false alarms, especially when class weighting encourages minority-stage predictions. High N1 recall is not enough if many W, N2 or REM epochs are incorrectly called N1. If a model never predicts a stage, its undefined precision is assigned zero rather than omitted from the mean.

4.2.3 Macro-Averaged Recall

Definition. Recall, or sensitivity, measures how many true examples of a stage are recovered:

$$R_c = \frac{TP_c}{TP_c + FN_c}, \qquad \text{Macro-Recall} = \frac{1}{K}\sum_{c=1}^{K} R_c. \tag{14}$$

Recall is included to expose missed stages. It complements precision: increasing the number of N1 predictions can recover more true N1 epochs while also introducing false positives. Macro-recall is retained in the results for explicit sensitivity reporting, but it is mathematically identical to the unadjusted BAcc in this five-class setting. These two columns therefore are not independent evidence of a model's advantage. Every reported full partition contains all five stages.

4.2.4 Per-Class F1 and Macro-F1

Definition. Stage F1 is the harmonic mean of precision and recall, and macro-F1 is the arithmetic mean of those five stage scores:

$$F1_c = \frac{2P_cR_c}{P_c + R_c} = \frac{2TP_c}{2TP_c + FP_c + FN_c}, \qquad \text{Macro-F1} = \frac{1}{K}\sum_{c=1}^{K} F1_c. \tag{15}$$

An undefined F1 is assigned zero. Macro-F1 is not obtained by taking the harmonic mean of macro-precision and macro-recall, because averaging and this nonlinear operation do not commute [14].

Why it selects checkpoints. The task requires useful detection of every sleep stage while controlling false alarms. Macro-F1 makes a weak stage visible and considers both types of error. The checkpoint rule is consequently the highest validation macro-F1, with BAcc, kappa and MGm as supporting measures. Selecting checkpoints by BAcc would answer a different question, emphasizing recall without the same direct precision trade-off; the selection rule is not changed after examining test rankings. The five per-class F1 columns show whether a strong aggregate score conceals a weak N1 or REM result.

4.2.5 Cohen's Kappa: Chance-Adjusted Agreement

Definition. Cohen's kappa compares observed agreement with the agreement expected if two independent label assignments retained the observed class marginals [15, 16]. Here the two label sources are the expert reference and the model prediction:

$$p_o = \frac{1}{N}\sum_{c=1}^{K} M_{cc} = \mathrm{Acc}, \qquad p_e = \sum_{c=1}^{K} \frac{n_c}{N}\frac{\hat{n}_c}{N}, \qquad \kappa = \frac{p_o - p_e}{1 - p_e}. \tag{16}$$

Perfect agreement gives $\kappa = 1$; agreement equal to this marginal-based expectation gives zero; a negative value indicates less agreement than expected. The denominator represents the maximum improvement over the expected agreement. It is not a correction obtained by subtracting the constant 1/5.

Why it is added. A model can agree frequently with the reference partly because some stages are common. Kappa supplies a chance-adjusted view of the full confusion matrix, complementing raw accuracy and the equal-stage F1 average. For the archived Transformer test result, $p_o = 0.798853$ and $p_e = 0.263095$, giving $\kappa = 0.7270$. This demonstrates how the marginal stage frequencies enter the result.

The unweighted form is used: W, N1, N2, N3 and REM are categorical labels, and their integer encoding does not define a defensible linear error distance. Kappa is influenced by prevalence and prediction marginals; it does not eliminate class imbalance, assess probability calibration or establish clinical acceptability. Universal qualitative labels such as "excellent" are therefore avoided.

4.2.6 Macro-Averaged Geometric Mean (MGm)

Definition. Specificity measures rejection of non-c epochs when stage c is treated as the positive class. The adopted MGm first forms a geometric mean for each one-versus-rest problem and then takes an arithmetic mean across stages:

$$S_c = \frac{TN_c}{TN_c + FP_c}, \qquad G_c = \sqrt{S_c R_c}, \qquad \boxed{\mathrm{MGm} = \frac{1}{K}\sum_{c=1}^{K} \sqrt{S_c R_c}}. \tag{17}$$

This is the mean of five separate class-wise G-means, consistent with computing the unaveraged one-versus-rest scores before averaging them [17]. It differs from both $(\prod_c R_c)^{1/K}$ and $\sqrt{\text{Macro-Specificity}\,\text{Macro-Recall}}$. Computing a geometric mean after macro-averaging therefore would not reproduce the measure used here.

Why it is added. MGm jointly measures recovery of a stage and rejection of all other stages. For each class, the geometric mean falls to zero if either sensitivity or specificity is zero, discouraging a classifier that obtains a strong value for only one of them. The macro average gives equal weight to the five stage-specific trade-offs. It complements F1, which uses false positives and false negatives without a true-negative term.

For the Transformer test N1 column, $TP = 1209$, $FN = 872$, $FP = 2170$ and $TN = 20681$. Thus $R_{\mathrm{N1}} = 0.5810$, $S_{\mathrm{N1}} = 0.9050$ and $G_{\mathrm{N1}} = 0.7251$. Averaging all five G_c values gives MGm = 0.8512, despite accuracy = 0.7989. MGm is not an accuracy percentage.

Its specificity term can be high when the other four stages provide many negatives, and it does not replace minority-stage precision. If one stage is never recognized, that stage contributes zero; the arithmetic macro average need not itself be zero.

4.2.7 Reporting Priority and Interpretation

Accuracy and macro-F1 remain the required headline classification measures; validation macro-F1 alone selects the checkpoint. BAcc summarizes stage-balanced sensitivity, kappa adjusts agreement for marginals, MGm combines sensitivity and specificity, and per-stage F1 identifies concentrated weaknesses. These are complementary views of the same predictions, not seven independent experiments. AUROC is excluded from the final comparison. None of these hard-label measures establishes calibrated probabilities, and a small numerical difference without uncertainty analysis is not evidence of a reliable generalization advantage.

4.3 Three Distinct Result Summaries

1. **Fold mean ± SD:** compute metrics for each selected validation checkpoint and take an unweighted mean across five folds. SD uses denominator five, matching the saved summaries. It describes variation across these folds, not a confidence interval or repeated-seed uncertainty.

2. **Pooled validation:** concatenate predictions for each model's own validation fold. Each of the 170,537 development epochs contributes once. Metrics are recomputed from the combined counts; nonlinear metrics need not equal their fold means. This is pooling, not ensembling, and remains validation-based because these labels selected the checkpoints.

3. **Held-out test ensemble:** apply all five fold checkpoints to the same 24,932 test epochs and average probabilities:

$$\bar{p}(c \mid X) = \frac{1}{5}\sum_{k=1}^{5} \mathrm{softmax}(f_{\theta_k}(X))_c, \qquad \hat{y} = \arg\max_c \bar{p}(c \mid X). \tag{18}$$

   Every checkpoint excludes the test participants from training and validation. One aggregate test score is reported, without attaching the development-fold SD to it.

These summaries answer different questions and are all retained; the reporting method is not selected according to which gives the largest score. A difference between pooled validation and ensemble test performance does not quantify the benefit of ensembling, because both the evaluated participants and predictor differ.

4.4 Comparison Hypotheses and Controlled Experiments

Model comparison. The hypothesis is that waveform-aware temporal modelling or pretrained EEG features improve macro-F1 over the simple CNN. Fixed factors are eligi-

ble epochs, participant partitions, class mapping, common waveform preparation, class-weight rule and evaluation definitions. The changed factor is the complete model configuration. Learning rates, batch sizes and effective channel inputs also differ, so this is not an isolated test of architecture or pretraining. Higher development macro-F1 is the primary comparison criterion; test results characterize generalization, with cost considered alongside accuracy.

Preparation-recipe sensitivity study. A completed AttnSleep experiment compares the common filtered/scaled preparation with a native-rate preparation at 100 Hz in microvolts, without the additional common bandpass or upsampling. Fixed factors are the AttnSleep architecture, Fpz-Cz input, participant memberships, weighting formula, optimizer settings, batch size, epoch cap and selection rule. The changed factor is the preparation recipe as a whole. The question is whether the shared recipe causes a material observed reduction in stage-balanced performance; five-fold development macro-F1 is the decision measure, with test metrics as a descriptive check.

The native recipe retains ten additional W epochs in development because of boundary-selection differences. Test epoch identities, order and labels are identical, and all validation participant memberships match. This is a recipe-level sensitivity study, not a perfectly isolated filter, amplitude or sample-rate ablation. The initial seed is shared but later fold random streams need not match after different stopping times. No statistical equivalence or causal attribution to an individual preprocessing step is claimed.

Inference-only ensemble ablation. To isolate a component of the final prediction procedure, the five existing Transformer checkpoints are re-evaluated on exactly the same held-out epochs. The hypothesis is that equal-weight soft voting improves macro-F1 relative to the average performance of using one constituent predictor. The changed factor is using one checkpoint versus averaging all five probability vectors. The checkpoint pool, input tensor, labels, sample order, CPU float32 precision, batch size and metric definitions are fixed. Each single model is evaluated, rather than choosing the best one using test labels. No parameters are retrained and no ensemble weights are fitted.

The decision criterion, specified for this additional analysis, is a positive difference between ensemble macro-F1 and the mean of the five single-model macro-F1 scores. Accuracy, BAcc, kappa and MGm provide secondary checks; a paired participant bootstrap describes uncertainty conditional on the fixed checkpoints. Inference requires five forward passes rather than one. This is a post-hoc evaluation-factor ablation on an already inspected test set, not a fresh confirmatory test or an ablation of learned architecture components. It isolates the aggregation rule for a fixed pool of trained models; it cannot establish the effect of removing attention, positional encoding or pretraining.

V Experimental Results

5.1 Five-Fold Development Performance

Model	Accuracy	BAcc	Macro-F1	κ	MGm
Custom CNN	0.7512 ± 0.0084	0.6699 ± 0.0186	0.6750 ± 0.0144	0.6551 ± 0.0121	0.7805 ± 0.0136
Transformer	$\mathbf{0.8056 \pm 0.0079}$	$\mathbf{0.7431 \pm 0.0047}$	$\mathbf{0.7498 \pm 0.0039}$	$\mathbf{0.7317 \pm 0.0108}$	$\mathbf{0.8358 \pm 0.0029}$
AttnSleep	0.7981 ± 0.0094	0.7418 ± 0.0078	0.7451 ± 0.0053	0.7222 ± 0.0112	0.8347 ± 0.0045
CBraMod	0.7906 ± 0.0101	0.7374 ± 0.0070	0.7349 ± 0.0070	0.7125 ± 0.0129	0.8302 ± 0.0049

Table 5: *Validation mean ± population SD; bold marks the highest mean per metric.*

Model	Macro-precision	Macro-recall
Custom CNN	0.6823 ± 0.0090	0.6699 ± 0.0186
Transformer	$\mathbf{0.7625 \pm 0.0059}$	$\mathbf{0.7431 \pm 0.0047}$
AttnSleep	0.7534 ± 0.0037	0.7418 ± 0.0078
CBraMod	0.7388 ± 0.0086	0.7374 ± 0.0070

Table 6: *Validation macro-precision and macro-recall (mean ± SD).*

Model	W	N1	N2	N3	REM
Custom CNN	0.9025 ± 0.0124	0.3329 ± 0.0251	0.8011 ± 0.0064	0.7911 ± 0.0326	0.5471 ± 0.0268
Transformer	$\mathbf{0.9288 \pm 0.0102}$	$\mathbf{0.4510 \pm 0.0150}$	$\mathbf{0.8378 \pm 0.0115}$	0.8064 ± 0.0193	$\mathbf{0.7251 \pm 0.0155}$
AttnSleep	0.9164 ± 0.0117	0.4432 ± 0.0131	0.8372 ± 0.0155	0.8174 ± 0.0336	0.7112 ± 0.0113
CBraMod	0.9220 ± 0.0107	0.4226 ± 0.0259	0.8201 ± 0.0117	$\mathbf{0.8175 \pm 0.0278}$	0.6925 ± 0.0272

Table 7: *Per-stage validation F1 (mean ± SD); bold marks the highest mean.*

The temporal Transformer leads all seven displayed aggregate validation measures, although BAcc and macro-recall are the same quantity. It does not lead every stage: CBraMod has the highest mean N3 F1 (0.8175), only 0.0001 above AttnSleep. These small differences are descriptive, not significance claims. The temporal Transformer has the highest mean validation macro-F1 (0.7498), improving on the CNN by 0.0748, or 7.48 percentage points. AttnSleep improves on the CNN by 7.01 points and CBraMod by 5.99 points. Pretraining does not provide the strongest result under the present head and optimization settings. The CNN's low initial learning rate and different convergence pattern also prevent interpreting the gap as an architecture-only effect.

Model	Accuracy	BAcc	Macro-F1	κ	MGm	Macro-P	Macro-R
Custom CNN	0.7512	0.6698	0.6756	0.6554	0.7809	0.6821	0.6698
Transformer	**0.8058**	**0.7436**	**0.7504**	**0.7321**	**0.8365**	**0.7611**	**0.7436**
AttnSleep	0.7982	0.7420	0.7456	0.7226	0.8351	0.7523	0.7420
CBraMod	0.7908	0.7373	0.7362	0.7129	0.8311	0.7367	0.7373

Table 8: *Pooled validation over 170,537 epochs; bold marks the highest values.*

Model	W	N1	N2	N3	REM
Custom CNN	0.9026	0.3344	0.8011	0.7928	0.5473
Transformer	**0.9294**	**0.4518**	**0.8380**	0.8070	**0.7259**
AttnSleep	0.9173	0.4433	0.8376	**0.8187**	0.7112
CBraMod	0.9225	0.4251	0.8202	0.8181	0.6951

Table 9: *Per-stage pooled validation F1; bold marks the highest values.*

The pooled ranking agrees with the fold-average ranking. Pooling gives more influence to larger folds, while fold means give each fold equal weight. Small numerical differences are therefore expected, especially for F1 and kappa. All tabulated hard-label metrics were independently checked against the saved confusion matrices and their class supports. CBraMod's pooled re-evaluation differs slightly from the sum of its original fold matrices (14 in the sum of absolute cell-count differences); its independently saved pooled result is retained. The other three pooled matrices match their fold sums exactly. The CBraMod discrepancy cannot be resolved from aggregate records alone.

5.2 Held-Out Participant Evaluation

Bold marks the highest value in each metric column; it does not indicate statistical significance.

Model	Accuracy	BAcc	Macro-F1	κ	MGm	Macro-P	Macro-R
Custom CNN	0.7634	0.7088	0.7063	0.6765	0.8097	0.7061	0.7088
Transformer	**0.7989**	**0.7666**	**0.7564**	**0.7270**	**0.8512**	**0.7595**	**0.7666**
AttnSleep	0.7970	0.7610	0.7506	0.7250	0.8468	0.7490	0.7610
CBraMod	0.7863	0.7596	0.7422	0.7123	0.8454	0.7357	0.7596

Table 10: *Test ensemble metrics on 24,932 epochs from 12 participants. Macro-P/R denote macro precision/recall.*

Model	W	N1	N2	N3	REM
Custom CNN	0.8500	0.3745	0.8320	0.8200	0.6550
Transformer	0.8763	**0.4429**	**0.8498**	0.8235	**0.7896**
AttnSleep	**0.8842**	0.4080	0.8473	**0.8362**	0.7773
CBraMod	0.8755	0.4106	0.8347	0.8247	0.7656

Table 11: *Per-stage F1 for the same test ensembles as Table 10.*

The temporal Transformer obtains accuracy 0.7989 and macro-F1 0.7564. Relative to the CNN, the gain in macro-F1 is 5.01 percentage points. Its macro-F1 exceeds AttnSleep by only 0.58 points and CBraMod by 1.42 points. The Transformer makes 5,015 errors, compared with 5,061 for AttnSleep, 5,329 for CBraMod and 5,900 for the CNN. The 46-error difference from AttnSleep is modest on 24,932 correlated epochs and is not evidence of statistical superiority.

AttnSleep has the highest W and N3 F1; the Transformer has the highest N1, N2 and REM F1. Thus the aggregate ranking hides useful stage-specific trade-offs. All scores in the main test comparison describe five-checkpoint ensembles, not a single deployed model. Section 5.4 separately evaluates individual Transformer checkpoints and the ensemble under matched inference conditions.

For the CNN–Transformer comparison, saved epoch-level test predictions permit a paired participant bootstrap. Resampling the 12 participants with replacement 2,000 times (seed 42), retaining all epochs of each sampled person, gives a percentile 95% interval of $[0.0373, 0.0621]$ for the Transformer-minus-CNN macro-F1 difference. This supports the observed improvement over this baseline within the sampled test cohort. It is conditional on the trained models and only 12 people; it does not quantify retraining variability or justify significance claims against AttnSleep.

5.3 Learning Behaviour and Selected Epochs

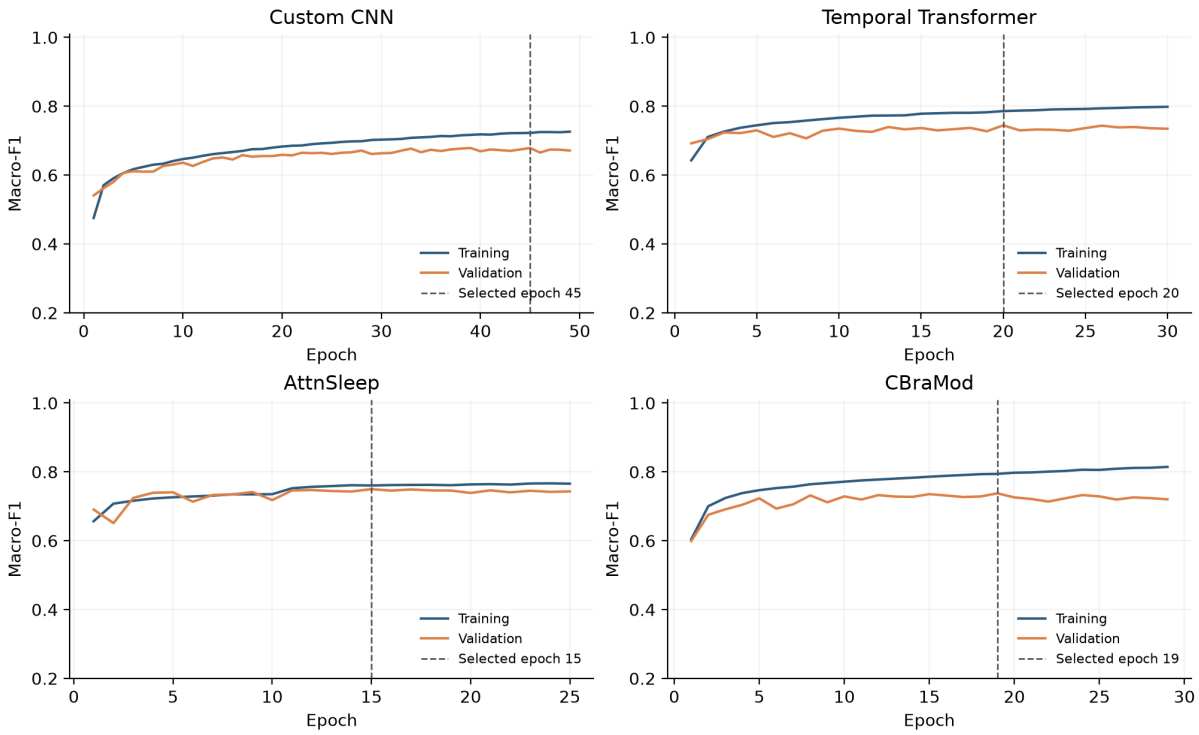


Figure 5: *Fold 1 macro-F1 curves; dashed lines mark selected checkpoints.*

Training metrics are collected during weight updates; validation metrics are measured in evaluation mode.

Model	Selected epochs (folds 1–5)	Completed epochs (folds 1–5)
Custom CNN	45, 44, 33, 33, 49	49, 54, 42, 43, 59
Transformer	20, 15, 18, 24, 26	30, 23, 28, 33, 36
AttnSleep	16, 29, 12, 15, 14	26, 33, 22, 24, 24
CBraMod	19, 9, 15, 13, 23	29, 19, 25, 22, 28

Table 12: *Selected and completed training epochs for each fold.*

The CNN requires 42–59 epochs, whereas CBraMod stops after 19–29 and the temporal Transformer after 23–36. AttnSleep varies from 22 to 33. Rising training performance beyond the selected validation epoch is consistent with diminishing generalization benefit; it does not justify choosing the final epoch automatically. A later small improvement can update the checkpoint without resetting the early-stopping counter, explaining why stopping need not occur exactly ten epochs after the selected checkpoint.

5.4 Controlled Ablation of Ensemble Aggregation

The inference ablation described in Section 4.4 uses all 24,932 test epochs and the five already selected Transformer checkpoints. Table 13 reports every single-model score and their equal-weight soft-voting result. All are recomputed in CPU float32 with batch size 64 and four CPU threads, so the within-table comparison holds numerical precision and execution conditions fixed.

Predictor	Accuracy	BAcc	Macro-F1	κ	MGm
Fold 1 alone	0.7866	0.7527	0.7448	0.7102	0.8422
Fold 2 alone	0.7924	0.7553	0.7502	0.7177	0.8444
Fold 3 alone	0.7841	0.7466	0.7360	0.7074	0.8366
Fold 4 alone	0.7858	0.7566	0.7431	0.7102	0.8441
Fold 5 alone	0.7849	0.7555	0.7412	0.7096	0.8435
Single-model mean	0.7867	0.7533	0.7431	0.7110	0.8422
Single-model SD	0.0029	0.0036	0.0047	0.0035	0.0029
Five-model soft vote	**0.7989**	**0.7666**	**0.7565**	**0.7271**	**0.8512**

Table 13: *Inference-only Transformer ablation. Single-model SD measures variation between fixed predictors on the same test set.*

Soft voting increases macro-F1 from the mean individual score 0.743078 to 0.756452: an absolute gain of 0.013374, or 1.34 percentage points. Accuracy improves by 1.22 points, and the ensemble exceeds each constituent model on all five displayed measures. A paired bootstrap that resamples the twelve test participants 2,000 times, preserving all of each person's epochs, gives a percentile 95% interval of $[0.0101, 0.0166]$ for ensemble-minus-mean-individual macro-F1. This interval is conditional on these fixed checkpoints and participants; it does not capture retraining variability.

The observed positive difference supports the aggregation hypothesis for this checkpoint pool. It is consistent with the models making partially different errors, allowing probability averaging to correct some individual decisions. Five forward passes are required for each epoch, and five parameter sets must be retained. Sequential evaluation allows one model at a time in accelerator memory, so fivefold forward-pass work does not imply fivefold peak accelerator memory. No new training was required for this comparison.

The recomputed ensemble differs from the archived mixed-precision labels on three of 24,932 epochs. Its macro-F1 is 0.756452 versus the original 0.756409, a difference of approximately 0.000043. The main comparison keeps the original recorded result; only this ablation uses the matched CPU reconstruction. Neither an individual fold nor an

ensemble weighting was chosen using test performance. Nevertheless, this is a post-hoc study on previously inspected participants, and it establishes an inference-aggregation effect rather than a causal benefit from a particular learned architectural component.

5.5 Preparation-Recipe Sensitivity Results

This sensitivity analysis retains the original paired preparation experiments. Its common-recipe run is separate from the newly completed AttnSleep rerun used in the main comparison; the rerun was performed to complete checkpoint and prediction provenance, not to select a better test score.

AttnSleep evaluation	Common recipe	Native-rate recipe
Development epochs	170,537	170,547
Validation accuracy, mean ± SD	0.7958 ± 0.0124	0.7939 ± 0.0099
Validation macro-F1, mean ± SD	0.7441 ± 0.0065	0.7407 ± 0.0084
Pooled validation macro-F1	0.744570	0.741307
Test accuracy	0.796446	0.795484
Test macro-F1	0.750965	0.750480
Test kappa	0.725067	0.723202

Table 14: *AttnSleep preparation-recipe comparison; both test rows use five-model soft voting.*

The common recipe is higher by 0.34 percentage points in mean validation macro-F1 and by only 0.049 points in test macro-F1. This provides no observed evidence of a large performance penalty from sharing the waveform representation. It does not establish equivalence, nor demonstrate that filtering or scaling individually improves accuracy. The shared preparation is retained for consistency and reuse. This within-project comparison must not be interpreted as outperforming the author's published 20-fold subject-wise benchmark, which evaluates a different training/test protocol [10].

5.6 Model Size and Training Cost

Model	Trainable params	Epochs	Median s/epoch	Total h
Custom CNN	523,237	247	34.44	2.369
Transformer	595,973	150	81.17	3.399
AttnSleep	522,805	129	35.70	1.241
CBraMod	14,645,605	123	98.32	3.360

Table 15: *Model sizes and total RTX 3090 training/validation time across five folds. CBraMod also has 200 fixed parameters.*

Timers cover training, validation and metric calculation per epoch; they exclude preprocessing, setup, plotting, checkpoint loading and later ensemble evaluation. All four main runs used RTX 3090 GPUs, but batch sizes differ and the host, storage and resource

contention were not matched. These are observed configuration costs, not isolated architecture throughput measurements. The completed AttnSleep rerun required 1.24 hours over five folds, versus 3.40 hours for the temporal Transformer: approximately 2.74 times less logged training time for a test macro-F1 difference of 0.58 points. Parameter count alone does not predict this runtime difference.

5.7 Inference Cost

Model	Batch 1 (ms)	Batch 64 (ms)	ms/epoch at batch 64
Custom CNN	0.74	67.93	1.06
Transformer	4.15	224.17	3.50
AttnSleep	4.70	359.62	5.62
CBraMod	11.15	241.49	3.77

Table 16: *Single-model CPU inference benchmark; one epoch is one 30-second EEG sample.*

To supplement the unrecorded GPU inference timing, a separate CPU forward-pass benchmark uses the same current architectures on an Apple M3 Pro CPU, four threads, float32, Python 3.13.15, PyTorch 2.14.0, evaluation mode and disabled gradients. Inputs are already resident in memory; each batch size receives five warm-ups followed by 20 timed forwards, summarized by the median. This measures architecture execution with initialized task weights, not a re-evaluation of trained checkpoints, and excludes loading, preprocessing and probability aggregation. It must not be interpreted as RTX 3090 deployment latency. The different CPU ordering illustrates why measured hardware and batch size matter. The five-model ensemble performs five forwards per input; shared storage or sequential checkpoint loading can change memory use and overhead, but does not remove that extra computation.

VI Analysis and Error Analysis

6.1 Stage-Specific Strengths and Weaknesses

N1 is the weakest stage for every model: test F1 ranges from 0.3745 to 0.4429. N3 is less frequent in the full dataset but is classified much better. Therefore imbalance alone does not explain the difficulty. For the Transformer, N1 recall is 0.5810 and precision is 0.3578. It predicts N1 for 3,379 epochs even though only 2,081 true N1 epochs exist; 2,170 N1 predictions are false positives. Better minority recall is accompanied by substantial false alarms.

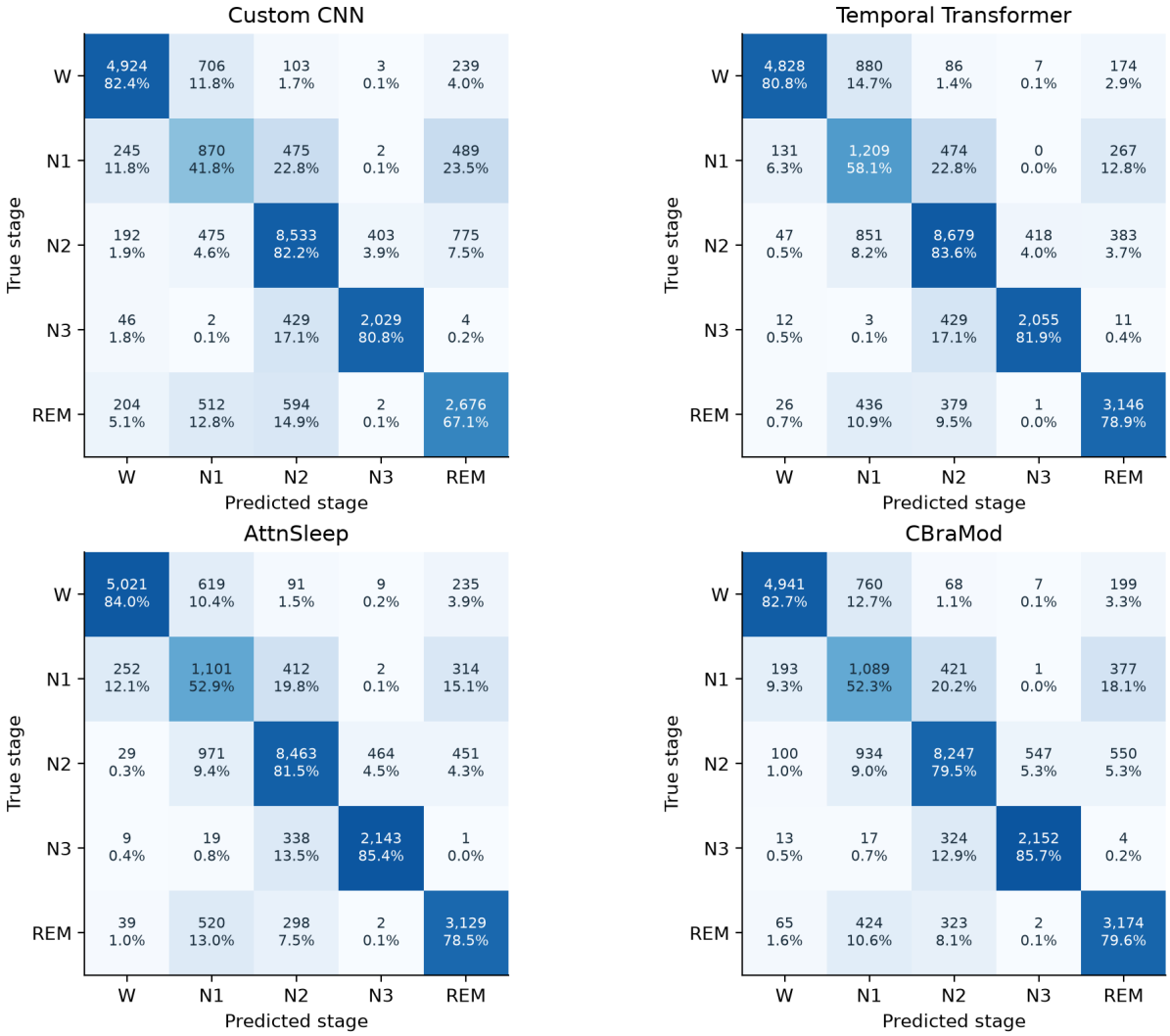


Figure 6: *Test ensemble confusion matrices, showing counts and true-stage row percentages. Colour indicates row percentage.*

6.2 Error Taxonomy

True → predicted	Count	% of true stage	% of all errors
W → N1	880	14.73	17.55
N2 → N1	851	8.20	16.97
N1 → N2	474	22.78	9.45
REM → N1	436	10.93	8.69
N3 → N2	429	17.09	8.55
N2 → N3	418	4.03	8.33

Table 17: *Largest Transformer error groups; the final column uses all 5,015 errors as its denominator.*

The three largest directed confusion groups account for 43.97% of Transformer errors. Similar ambiguity occurs in the other models: AttnSleep's largest group is N2 predicted

as N1 (932 epochs), and the CNN frequently exchanges N2 and REM (775 N2-to-REM and 594 REM-to-N2 cases). These patterns support investigating stage boundaries and temporal context, but aggregate counts cannot establish that a particular EEG event or transition caused an error.

Potential improvements include context spanning neighbouring 30-second epochs, better-balanced operating points for N1, and controlled comparisons of class weighting. Adding EOG could be studied for REM discrimination, but it would change the input modality and is outside the present experiment. Any such changes should be selected on development data and assessed on a new untouched participant evaluation if the current test results guide the change.

6.3 Correct, Difficult-Correct and Failure Examples

Figure 7 links actual held-out waveforms to saved ensemble predictions. Candidate pools consist of 64 evenly spaced test examples in each of three groups: correctly classified W, correctly classified N1, and N2 predicted as N1. Within these pools, the selected examples have respectively the largest, smallest and largest gap between the two highest mean probabilities. This defines difficulty operationally; it does not assert that the N1 case is the hardest epoch in the whole dataset.

Probabilities were recomputed from the five trained Transformer checkpoints in CPU float32 for these 192 candidates. All candidate argmax predictions matched the original saved predictions. The original full-test metrics are retained, rather than replaced by this illustrative subset evaluation. Waveforms are displayed in microvolts; bars show mean probabilities, with green identifying the true stage and the CNN prediction provided for comparison.

Confident correct. Participant SC404, recording SC4041, onset 58,890 s, is correctly classified as W by both models. The Transformer assigns W probability 0.9995. This illustrates a clear decision for one example, not calibrated certainty across the dataset.

Difficult but correct. For SC4041 at 32,070 s, the true stage is N1. The Transformer gives N1 probability 0.3559, REM 0.3416 and N2 0.2921: the top-two margin is only 0.0143. The CNN predicts REM. The disagreement between models and the probability pattern show an ambiguous decision boundary; they do not establish that the expert annotation is uncertain.

Confident error. For SC4551 at 30,330 s, the true stage is N2 but the Transformer predicts N1 with probability 0.8106; the CNN predicts REM. This is a concrete instance of the frequent N2-to-N1 error group. High confidence can accompany an incorrect stage, motivating calibration analysis and broader inspection of N1 false positives. No particular physiological event is asserted as the cause without expert review.

6.4 Interpretation of the Model Comparison

The custom temporal model retains per-second representations before attention, while the CNN compresses the waveform grid through repeated pooling. Its stronger N1 and REM F1 is compatible with the value of temporal modelling, but a controlled tokenization

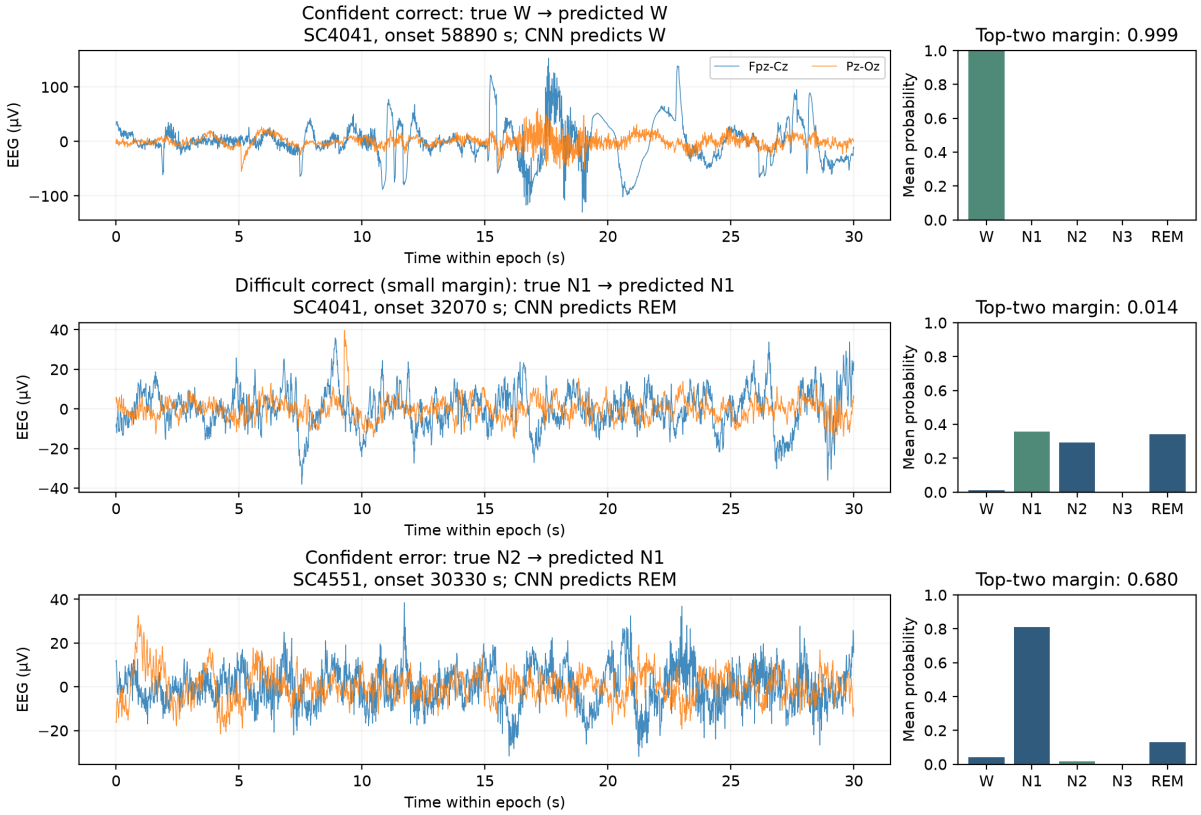


Figure 7: *Held-out examples: confident correct, low-margin correct, and confident incorrect.*

ablation would be needed to establish that mechanism. CBraMod's transfer features improve on the simple baseline, yet its large task head, pretrained montage differences and current fine-tuning schedule may limit the advantage. These are explanations to test, not demonstrated causes.

AttnSleep offers the strongest observed training-cost trade-off. The temporal Transformer provides a small improvement in aggregate test metrics with a similarly compact parameter count, at substantially greater logged training cost. No single model dominates every stage and every resource measure.

The ensemble ablation isolates a different contribution: combining the five fixed temporal models improves test macro-F1 relative to their average single-model score. It explains part of the final predictor's performance without attributing that gain to attention or pretraining. By contrast, the recipe study changes several preparation choices together, and the four-model comparison changes architecture and training settings. Keeping these three evidential scopes separate prevents a useful observed improvement from becoming an unsupported causal claim.

VII Reproducibility and Limitations

7.1 Attribution and Experiment Traceability

The custom CNN, temporal tokenization, task heads and common waveform workflow are project adaptations. AttnSleep's architecture is reused from its official MIT-licensed implementation; CBraMod's pretrained backbone and criss-cross attention likewise come from its MIT-licensed release [8, 12]. PyTorch supplies neural-network training, MNE and SciPy support EEG preparation, scikit-learn supplies grouped splitting and most metrics, and imbalanced-learn supplies class-wise geometric means.

Each fold retains the model parameters and persistent buffers of the selected validation checkpoint. Evaluation reconstructs the same architecture and uses those saved weights. These snapshots support inference but do not include the optimizer, scheduler and random states required for an exact interrupted-training restart. Dataset metadata retain recording and onset identities, and all main result tables can be reconciled with the corresponding fold or aggregate confusion matrices. The report's supporting evidence retains extracted measurements and source fingerprints; those fingerprints identify inspected materials, not the historical training commit.

The completed AttnSleep run records Python 3.12.15, PyTorch 2.14.1 with CUDA 13.2, its installed dependencies and an RTX 3090 directly from the training machine. The other three models retain the previously observed installation details and owner-confirmed RTX 3090 hardware. All twenty trained fold checkpoints strictly load into their corresponding architectures. All four test prediction records reproduce their reported confusion matrices; AttnSleep additionally supplies full validation and test probabilities, sample identities and per-epoch histories. Its pooled validation matrix equals the sum of the five selected-fold matrices. The original training commit remains unknown because its cloud execution did not use a Git checkout.

The AttnSleep cloud dataset serialization has a different file fingerprint from the available local tensor. All 195,469 sample identities, labels, participant splits, fold assignments and recorded preprocessing settings agree. The active AttnSleep adapter and author architecture are identical despite unrelated changes to the custom Transformer in the same source library. A CPU float32 reconstruction on 128 test examples reproduces all archived argmax predictions, with maximum probability difference 0.000339 relative to CUDA mixed precision. These checks support the tested reconstruction but do not establish bitwise equality of every waveform. Original cloud fingerprints and this verification scope are retained separately. Fine-tuned weights are kept locally, with documented dataset preparation and training instructions supporting reconstruction.

7.2 Data and Experimental Limits

Sleep Cassette, two bipolar EEG derivations, legacy annotations and annotation-informed wake cropping limit transfer to other cohorts and untrimmed recordings. No expert artefact audit or new rescoring was performed. The current full-data experiment must be reproduced with participant sampling disabled; the earlier quota-based preparation is a separate pilot configuration and does not regenerate these counts. Group separation

prevents participant overlap but does not remove the optimism introduced by repeated validation choices or test-set inspection.

Fold variability is not a substitute for participant-bootstrap uncertainty or repeated random seeds. The main models use different optimization settings and effective channel inputs, and the preparation study changes several signal-processing choices together. No superiority claim against published results is made. CPU inference timing is an architecture microbenchmark; matched GPU latency, peak memory and whole-pipeline deployment timing remain unmeasured. Historical commit identifiers remain unavailable; AttnSleep now includes its complete epoch-level probability archive and recorded runtime environment.

VIII Conclusion

The full-cohort experiment compares four EEG classifiers using 170,537 development epochs and 24,932 epochs from held-out participants. A compact temporal Transformer obtains the strongest observed aggregate test result, with macro-F1 0.7564, while AttnSleep achieves 0.7506 with substantially less logged training time. Fine-tuned CBraMod improves over the CNN baseline but does not outperform the compact temporal models in this configuration. N1 remains difficult across all methods.

The preparation sensitivity study supports practical reuse of a common waveform representation without an observed large loss for AttnSleep, while leaving the effect of individual preprocessing steps unresolved. A separate fixed-checkpoint ablation measures a 1.34-percentage-point macro-F1 gain from soft voting relative to the mean individual Transformer score, at five forward passes per epoch. The central outcome is a measured accuracy–cost comparison with participant separation and explicit evaluation definitions. The prediction-linked cases and paired participant bootstrap strengthen the analysis, while complete reproducibility records and broader independent participant evaluation remain necessary for stronger claims.

AI Usage Disclosure

OpenAI Codex assisted the requesting group member during October–November 2026 with waveform-pipeline adaptation, debugging, temporal Transformer design, metric checks, figures and report drafting. Representative requests concerned a waveform-native model and a final report aligned with the course requirements. Assistance affected the data, methods, results and analysis sections. Scores were checked against saved confusion matrices and available epoch predictions; metadata verified cohort counts and folds, while official sources verified attribution. The group confirmed the final architecture and GPU. Report preparation performed no training, but included a separate CPU benchmark, probability recovery for 192 illustrative test candidates, a full-test fixed-checkpoint Transformer ensemble ablation, and paired participant bootstraps. A prior EEG thesis informed the explanatory structure; its ASD-specific experiments and binary-task assumptions were not transferred to this report. The authors retain responsibility for reviewing all claims; named verification responsibilities and detailed prompts belong in the accompanying disclosure log.

A Participant Membership and Detailed Fold Results

Partition	Participant identifiers
Validation fold 1	SC400, SC405, SC409, SC410, SC421, SC425, SC434, SC443, SC444, SC447, SC456, SC471, SC473
Validation fold 2	SC401, SC415, SC418, SC426, SC437, SC441, SC448, SC450, SC451, SC452, SC454, SC462, SC476
Validation fold 3	SC408, SC422, SC427, SC430, SC446, SC453, SC458, SC463, SC464, SC465, SC474, SC480, SC482
Validation fold 4	SC402, SC406, SC412, SC432, SC433, SC436, SC438, SC442, SC449, SC457, SC459, SC460, SC475, SC477
Validation fold 5	SC407, SC416, SC419, SC420, SC423, SC424, SC429, SC440, SC445, SC461, SC466, SC467, SC470
Held-out test	SC403, SC404, SC411, SC413, SC414, SC417, SC428, SC431, SC435, SC455, SC472, SC481

Table 18: *Participant membership shared by the main comparison and native-rate sensitivity study.*

Model	Fold	Accuracy	BAcc	Macro-F1	κ	MGm
Custom CNN	1	0.7468	0.6740	0.6788	0.6498	0.7826
Custom CNN	2	0.7657	0.6962	0.6952	0.6756	0.8002
Custom CNN	3	0.7421	0.6659	0.6706	0.6450	0.7812
Custom CNN	4	0.7465	0.6385	0.6511	0.6435	0.7575
Custom CNN	5	0.7549	0.6750	0.6792	0.6617	0.7808
Transformer	1	0.8006	0.7361	0.7445	0.7246	0.8308
Transformer	2	0.8006	0.7429	0.7535	0.7243	0.8366
Transformer	3	0.7966	0.7410	0.7457	0.7201	0.8349
Transformer	4	0.8143	0.7449	0.7517	0.7425	0.8391
Transformer	5	0.8160	0.7505	0.7536	0.7469	0.8377
AttnSleep	1	0.7985	0.7499	0.7502	0.7234	0.8395
AttnSleep	2	0.7964	0.7480	0.7506	0.7187	0.8384
AttnSleep	3	0.7823	0.7365	0.7371	0.7033	0.8327
AttnSleep	4	0.8114	0.7291	0.7411	0.7366	0.8270
AttnSleep	5	0.8020	0.7454	0.7464	0.7291	0.8357
CBraMod	1	0.7855	0.7413	0.7376	0.7073	0.8339
CBraMod	2	0.7881	0.7474	0.7387	0.7094	0.8377
CBraMod	3	0.7798	0.7306	0.7319	0.6999	0.8287
CBraMod	4	0.7899	0.7283	0.7231	0.7084	0.8245
CBraMod	5	0.8096	0.7391	0.7434	0.7373	0.8261

Table 19: *Validation metrics at each selected checkpoint; per-stage summaries appear in Table 7.*

References

[1] National Heart, Lung, and Blood Institute, "How sleep works: Sleep phases and stages," 2022, national Institutes of Health. Accessed 10 November 2026. [Online]. Available: https://www.nhlbi.nih.gov/health/sleep/stages-of-sleep

[2] "CO3133: Deep Learning and Its Applications – Course Project Handbook," 2026, revision 14 September 2026. Report and reproducibility requirements: Sections 3–5; Assignment 2 requirements: Sections 20–23.

[3] B. Kemp, "Sleep-EDF Database Expanded," 2013, version 1.0.0; expanded archive described in 2018. Accessed 10 November 2026. [Online]. Available: https://physionet.org/content/sleep-edfx/1.0.0/

[4] B. Kemp, A. H. Zwinderman, B. Tuk, H. A. C. Kamphuisen, and J. J. L. Oberye, "Analysis of a sleep-dependent neuronal feedback loop: the slow-wave microcontinuity of the EEG," *IEEE Transactions on Biomedical Engineering*, vol. 47, no. 9, pp. 1185–1194, 2000. [Online]. Available: https://ieeexplore.ieee.org/document/867928/

[5] T. Pollard, B. E. Moody, L.-w. H. Lehman, B. Gow, C. Fernandes, C. Xie, A. Johnson, R. G. Mark, and T. Heldt, "PhysioNet as a global platform for biomedical research," *Nature Health*, vol. 1, no. 8, pp. 792–795, 2026. [Online]. Available: https://doi.org/10.1038/s44360-026-00096-z

[6] scikit-learn developers, "StratifiedGroupKFold: Group separation and approximate stratification," 2026, official documentation. Accessed 10 November 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

[7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://leon.bottou.org/papers/lecun-98h

[8] E. Eldele *et al.*, "AttnSleep: Official implementation," 2021, mIT-licensed architecture; adopted revision 6b4d2665884628c8a7bb09f36589a8ec0992f8e2. Accessed 10 November 2026. [Online]. Available: https://github.com/emadeldeen24/AttnSleep

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[10] E. Eldele, Z. Chen, C. Liu, M. Wu, C.-K. Kwoh, X. Li, and C. Guan, "An attention-based deep learning approach for sleep stage classification with single-channel EEG," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 29, pp. 809–818, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9417097

[11] J. Wang, S. Zhao, Z. Luo, Y. Zhou, H. Jiang, S. Li, T. Li, and G. Pan, "CBraMod: A criss-cross brain foundation model for EEG decoding," in *The Thirteenth International Conference on Learning Representations*, 2025,

author manuscript: https://arxiv.org/abs/2412.07236. [Online]. Available: https://openreview.net/forum?id=NPNUHgHF2w

[12] J. Wang *et al.*, "CBraMod: Official implementation and pretrained model," 2025, official MIT-licensed implementation and released pretrained weights. Accessed 10 November 2026. [Online]. Available: https://github.com/wjq-learning/CBraMod

[13] scikit-learn developers, "balanced_accuracy_score: Balanced accuracy," 2026, official documentation. Accessed 10 November 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.balanced_accuracy_score.html

[14] ——, "precision_recall_fscore_support: Per-class and averaged classification measures," 2026, official documentation. Accessed 10 November 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html

[15] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[16] scikit-learn developers, "cohen_kappa_score: Cohen's kappa," 2026, official documentation. Accessed 10 November 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.cohen_kappa_score.html

[17] imbalanced-learn developers, "geometric_mean_score: Multiclass and one-versus-rest geometric means," 2026, official documentation. Accessed 10 November 2026. [Online]. Available: https://imbalanced-learn.org/stable/references/generated/imblearn.metrics.geometric_mean_score.html